University of Liberal Arts Bangladesh

Department of Computer Science & Engineering

CSE 2201 : Algorithms

Lecture 10: Merge Sort

Atanu Shuvam Roy

Lecturer,
Dept. of CSE, ULAB, Dhaka

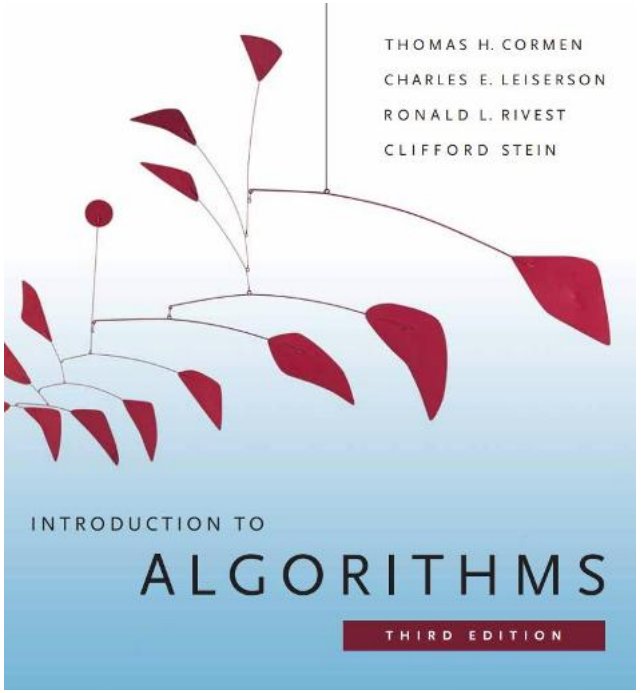
EMAIL:
atanu.shuvam@ulab.edu.bd

Class Code
GVWTHYJV

Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

Sorting problem

Ex. Student records in a university.

item →

key →

Chen	3	A	991-878-4944	308 Blair
Rohde	2	A	232-343-5555	343 Forbes
Gazsi	4	B	766-093-9873	101 Brown
Furia	1	A	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman

Sort. Rearrange array of N items into ascending order.

Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman
Chen	3	A	991-878-4944	308 Blair
Furia	1	A	766-093-9873	101 Brown
Gazsi	4	B	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Rohde	2	A	232-343-5555	343 Forbes

Today's Contents

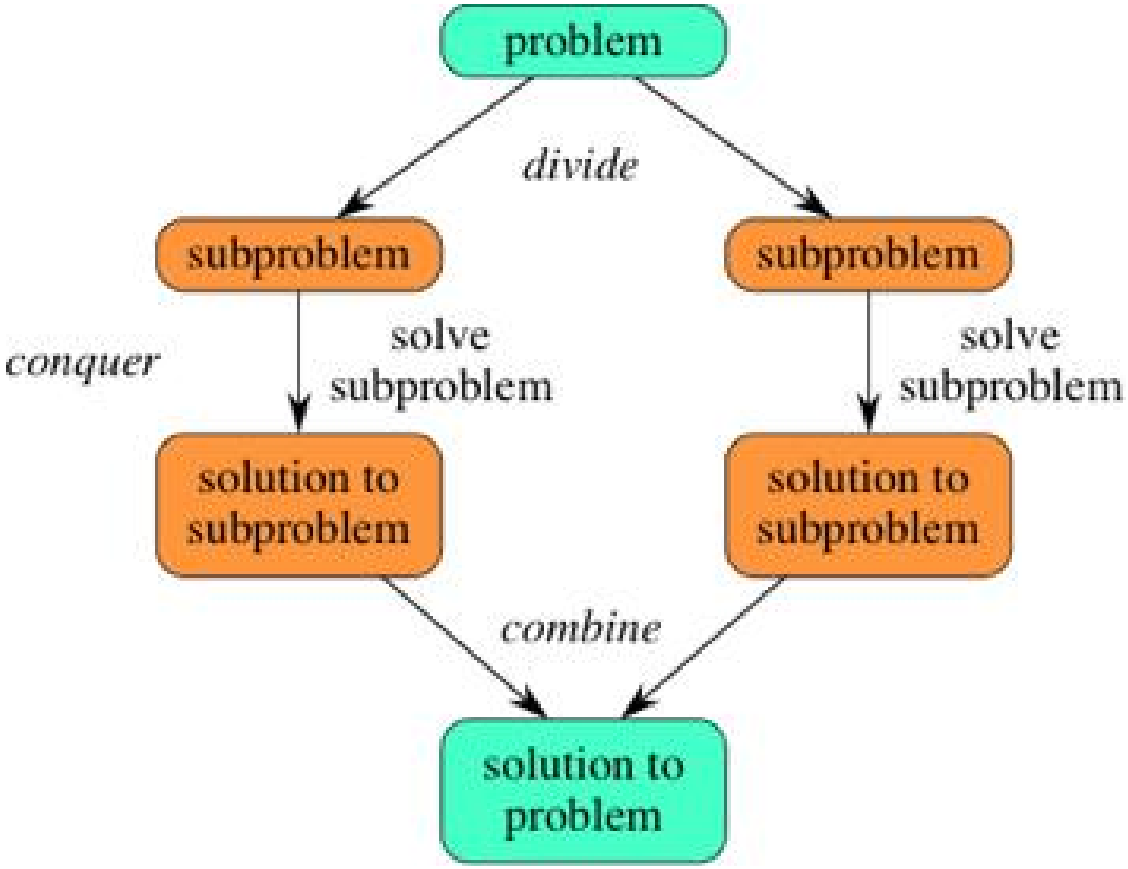
- Divide and Conquer Approach
- Merge Sort
- Complexity Analysis
- Questions for You!

Divide & Conquer Approach

The **divide-and-conquer** paradigm involves **three steps** at each level of the recursion:

1. **Divide** the problem into a number of **subproblems**.
2. **Conquer** the subproblems by solving them recursively. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner.
3. **Combine** the solutions to the subproblems into the solution for the original problem.

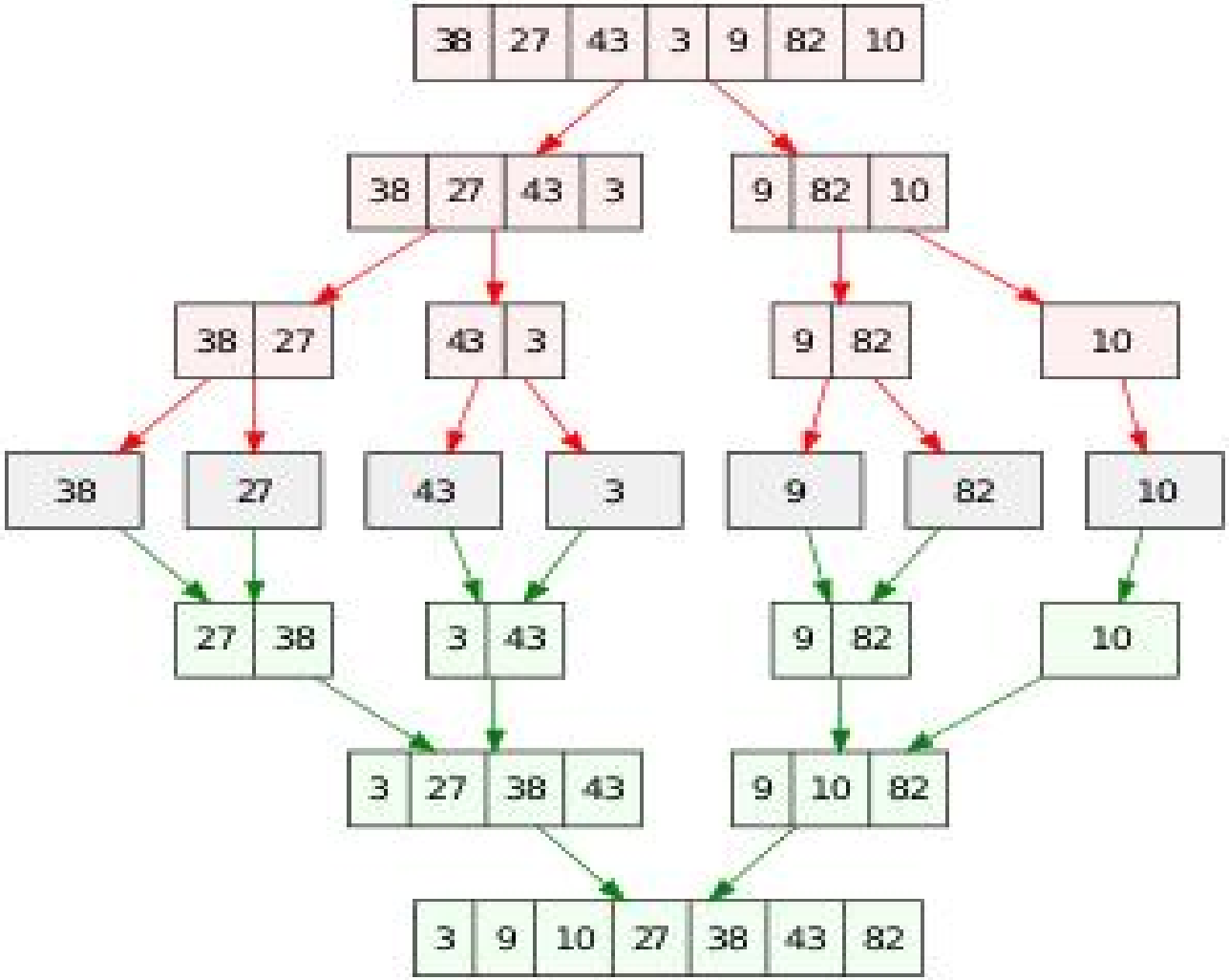
Divide & Conquer Approach



Merge Sort

- The **merge sort** algorithm closely follows the **divide-and-conquer** paradigm.
- Intuitively, it operates as follows.
  1. **Divide**: Divide the n-element sequence to be sorted into two subsequences of n/2 elements each.
  2. **Conquer**: Sort the two subsequences recursively using merge sort.
  3. **Combine**: Merge the two sorted subsequences to produce the sorted answer.

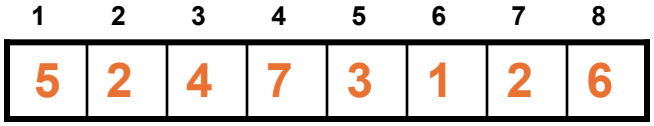
Example


38 27 43 3 9 82 10
38 27 43 3
9 82 10
38 27
43 3
9 82
10
38
27
43
3
9
82
10
27 38
3 43
9 82
10
3 27 38 43
9 10 82
3 9 10 27 38 43 82

p = first Index

r = last index

A[n]



$q \leftarrow \lfloor (p + r) / 2 \rfloor$

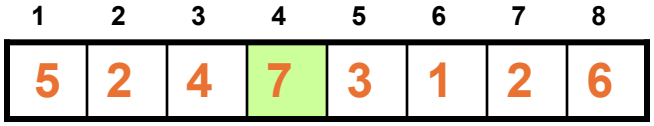
Left Array, L = A(1 , q)

Right Array, R = A[q + 1 , r]

p = first Index

r = last index

A[n]



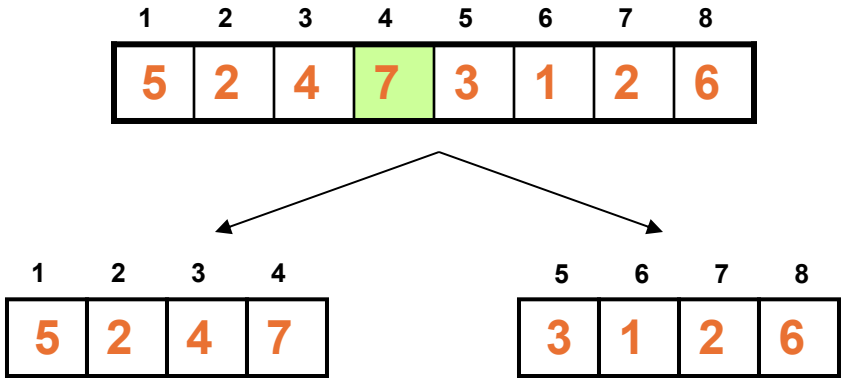
$q \leftarrow \lfloor (p + r) / 2 \rfloor$

q = (1 + 8) / 2 = 4

Left Array, L = A(1 , q)

Right Array, R = A[q + 1 , r]

Divide



p = first Index

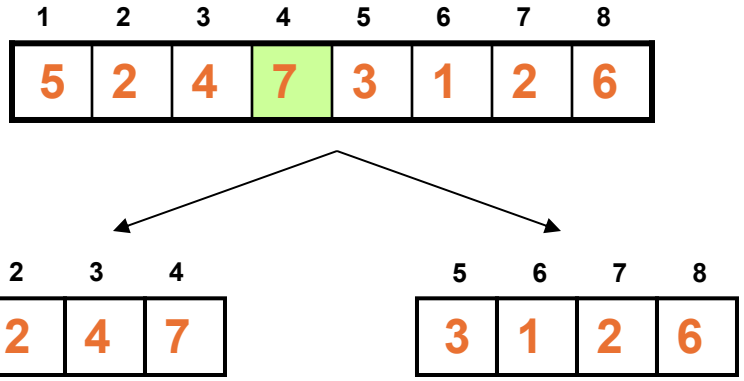
r = last index

$q \leftarrow \lfloor (p + r) / 2 \rfloor$

q = (1 + 8) / 2 = 4

$$q \leftarrow \lfloor (p + r) / 2 \rfloor$$

Divide

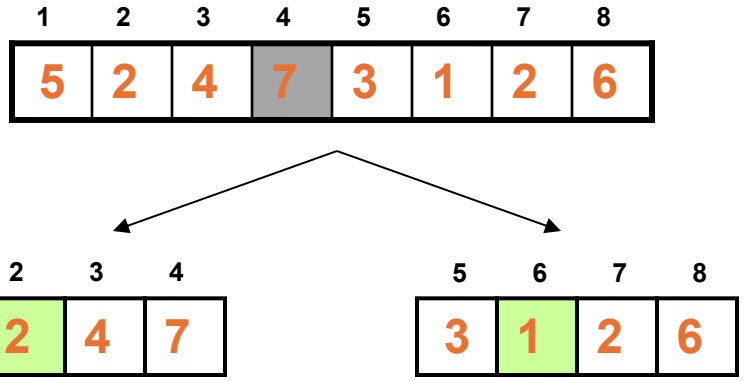


q = (1 + 4) /2
= 2

q = (5 + 8) /2
= 6

$$q \leftarrow \lfloor (p + r) / 2 \rfloor$$

Divide

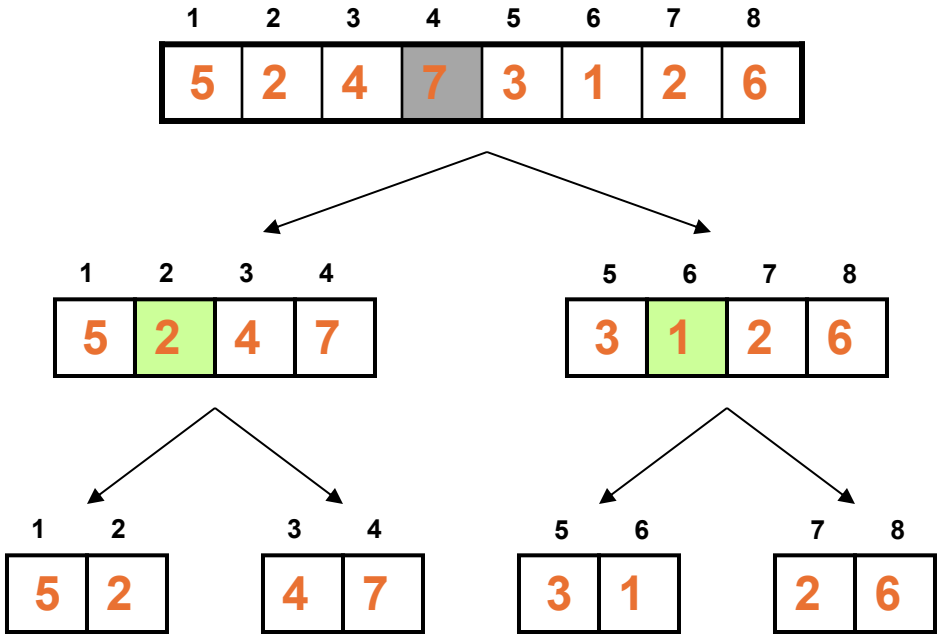


$q = (1 + 4) / 2$
$= 2$

$q = (5 + 8) / 2$
$= 6$

$$q \leftarrow \lfloor (p + r) / 2 \rfloor$$

Divide

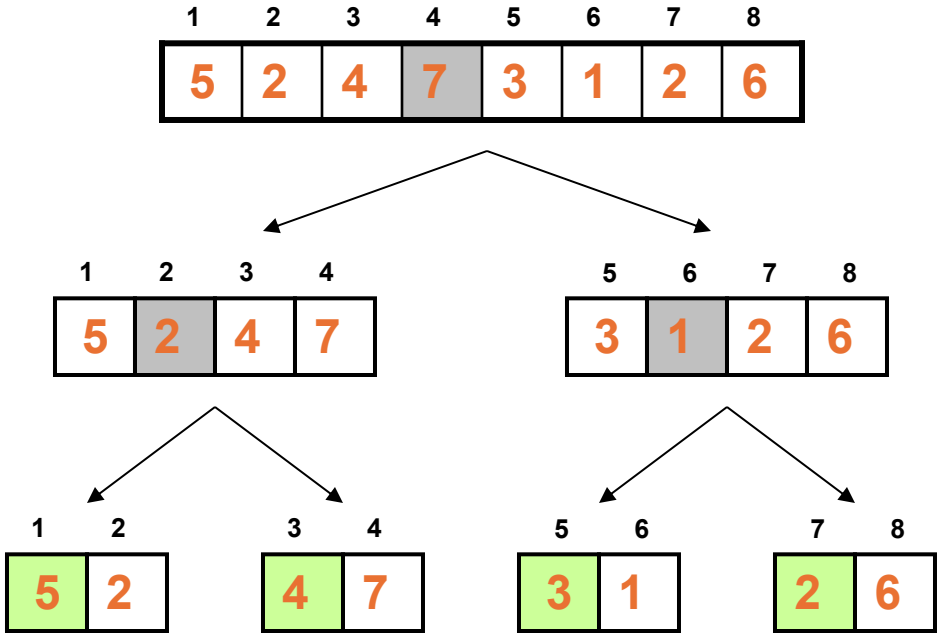


q = (1 + 4) /2
= 2

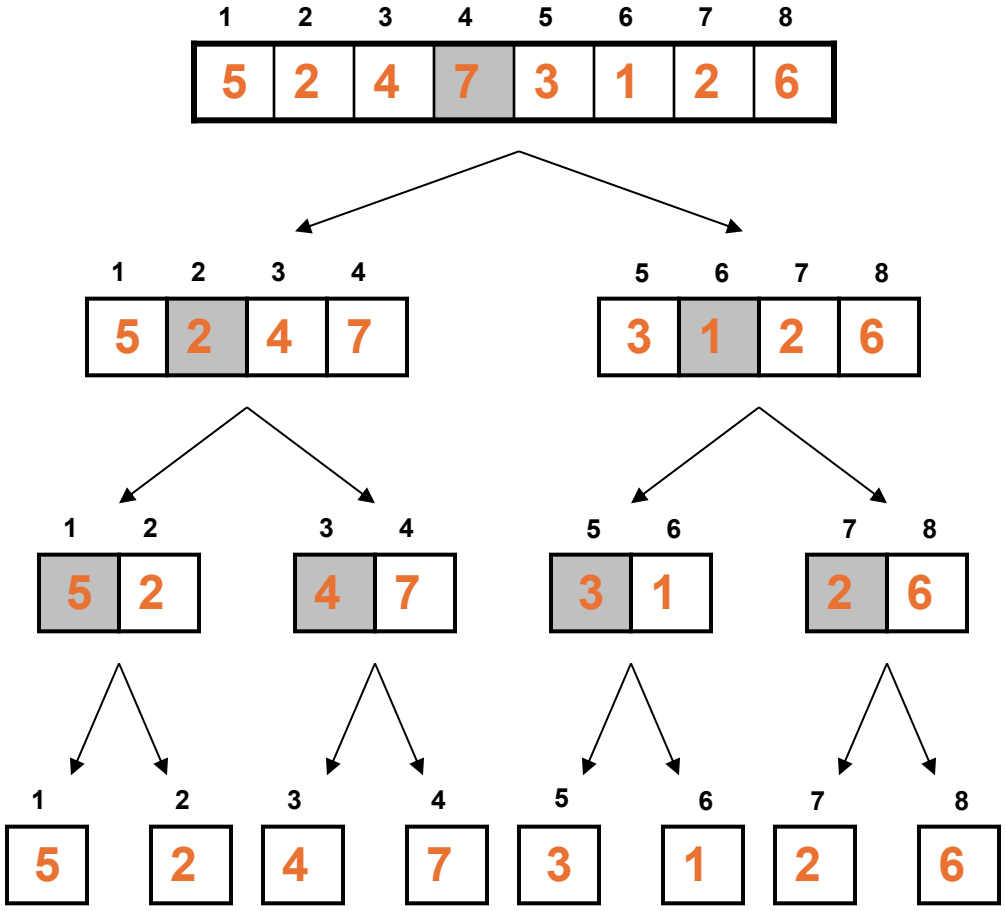
q = (5 + 8) /2
= 6

$$q \leftarrow \lfloor (p + r) / 2 \rfloor$$

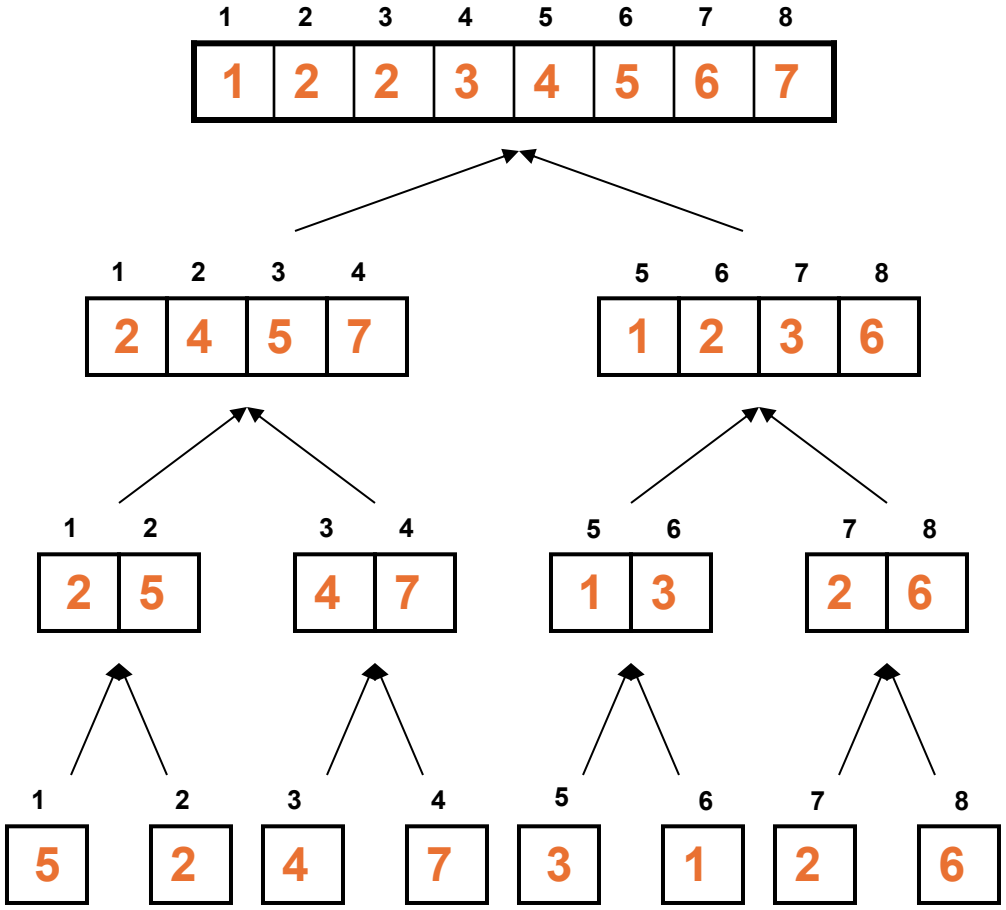
Divide



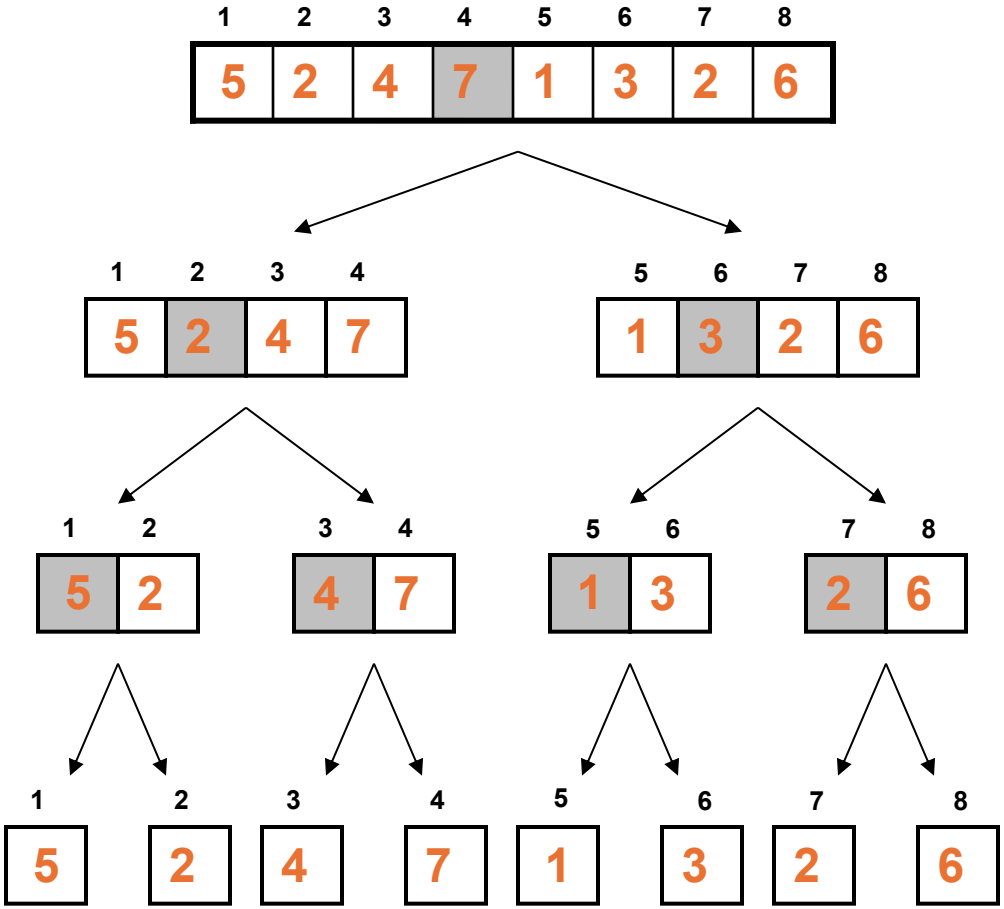
Divide


1 2 3 4 5 6 7 8
5 2 4 7 3 1 2 6
1 2 3 4
5 2 4 7
5 6 7 8
3 1 2 6
1 2
5 2
3 4
4 7
5 6
3 1
7 8
2 6
1
5
2
3
4
7
5
3
6
1
7
2
8
6

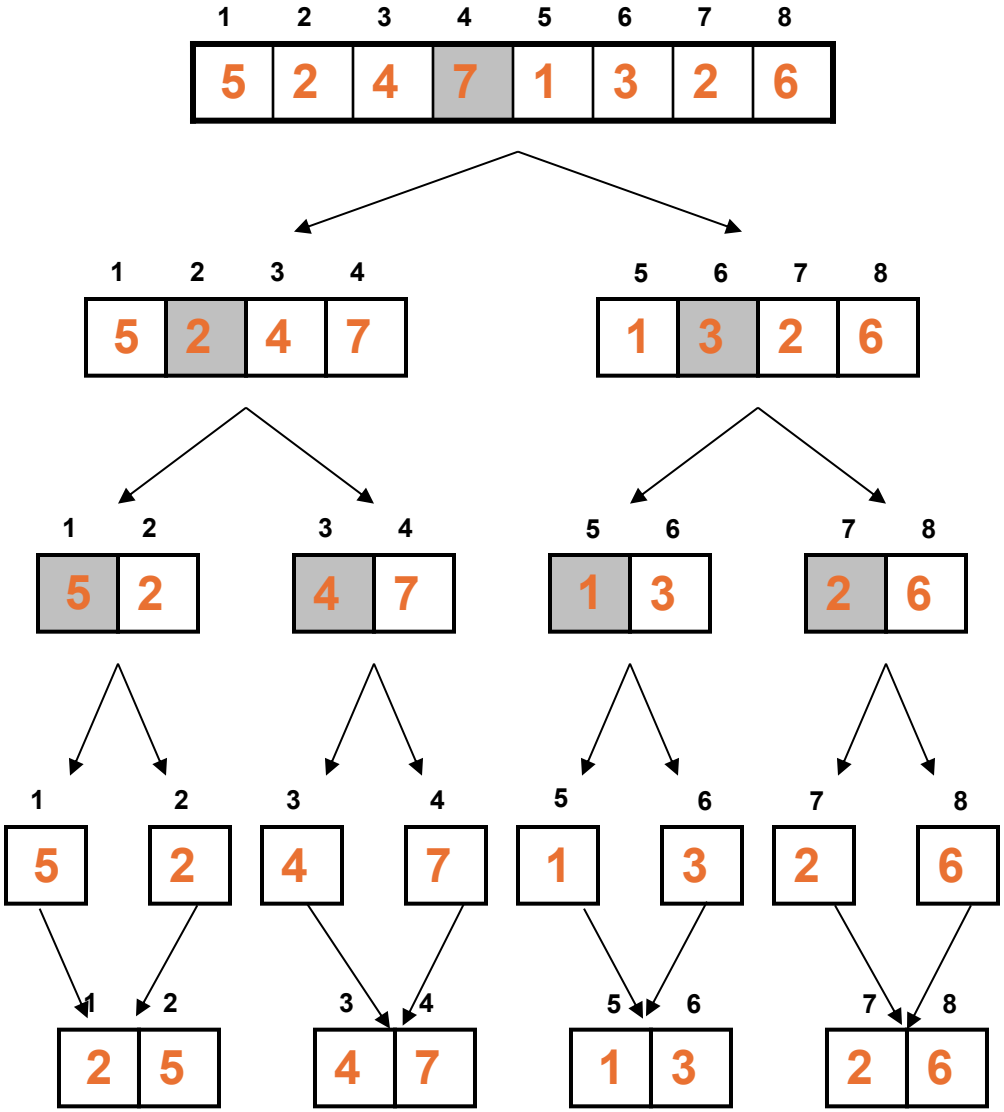
Conquer and Merge



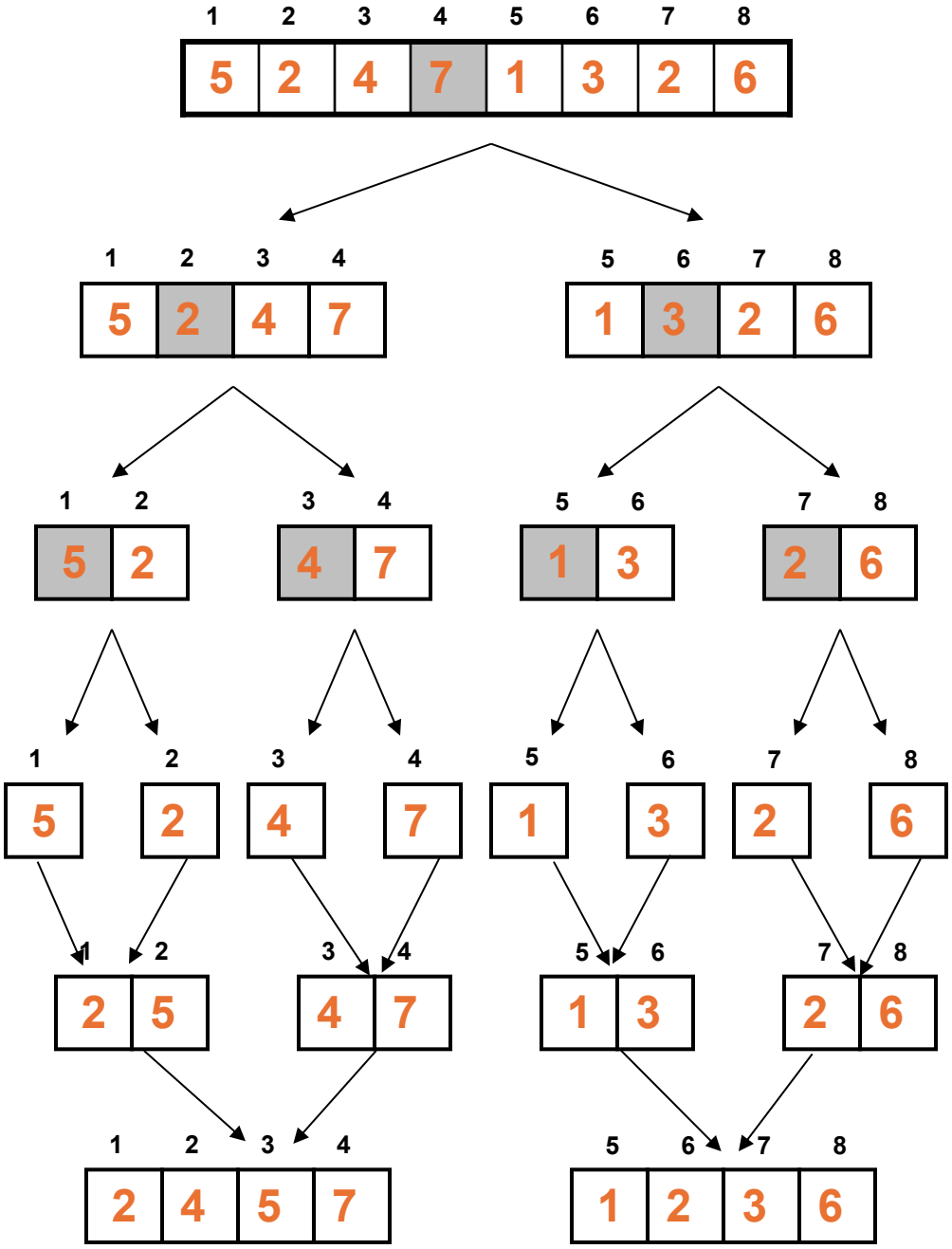
Conquer and Merge



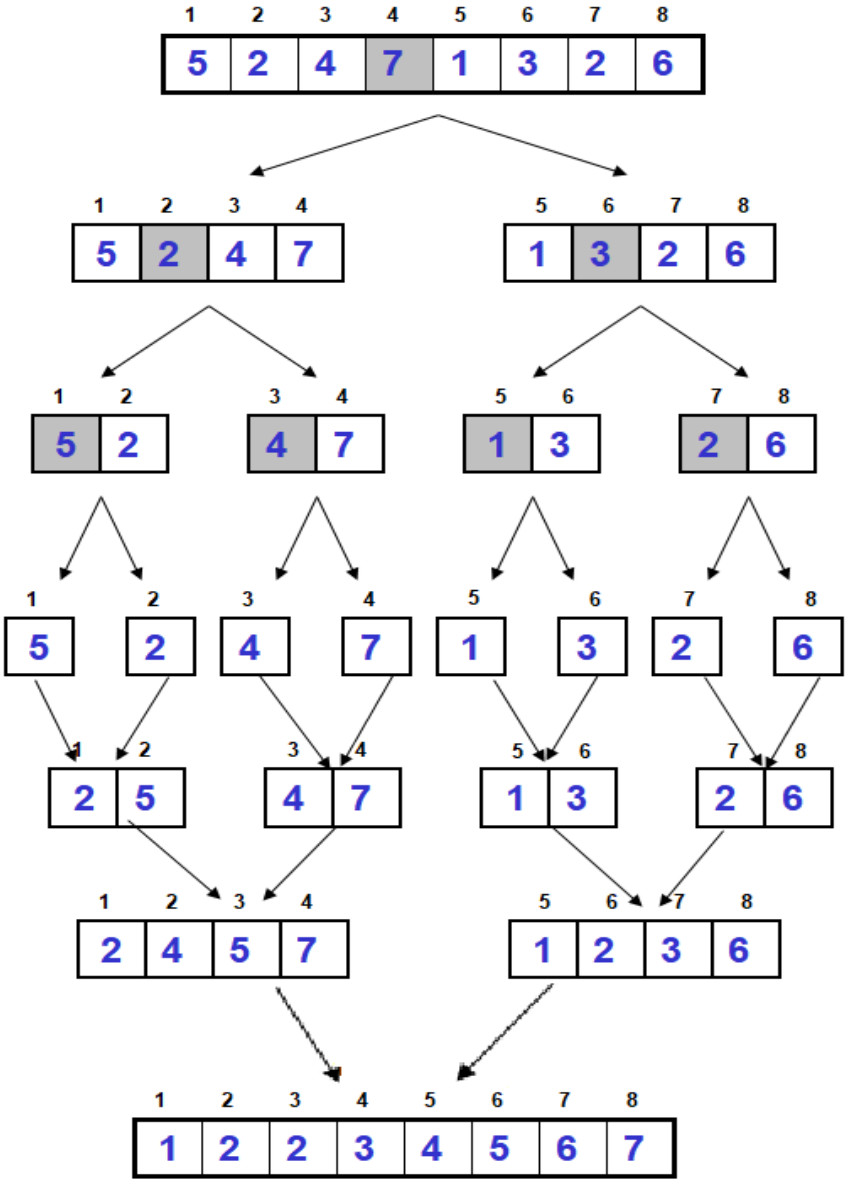
Conquer and Merge

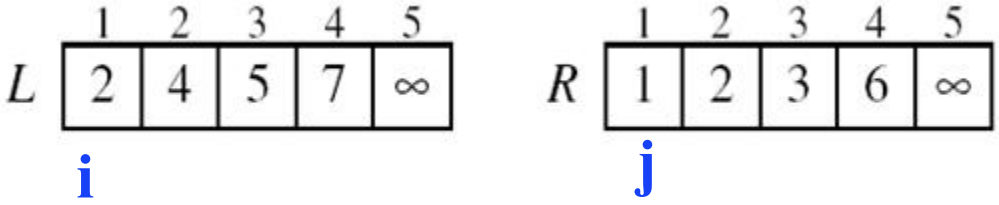


Conquer and Merge

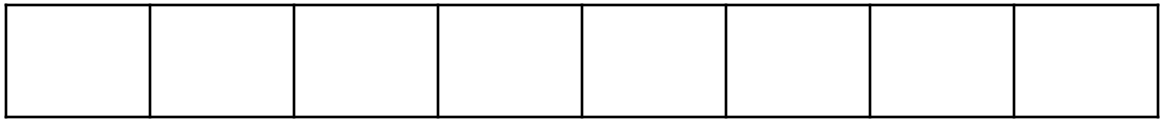


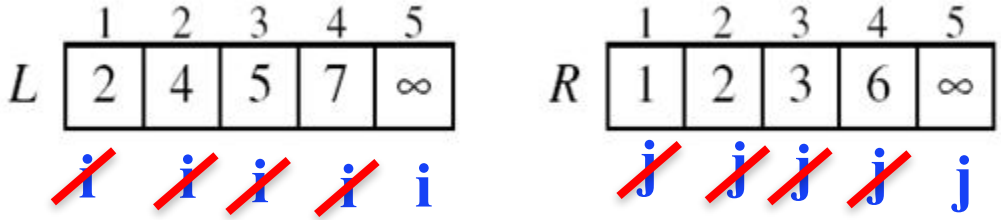
Conquer and Merge





A



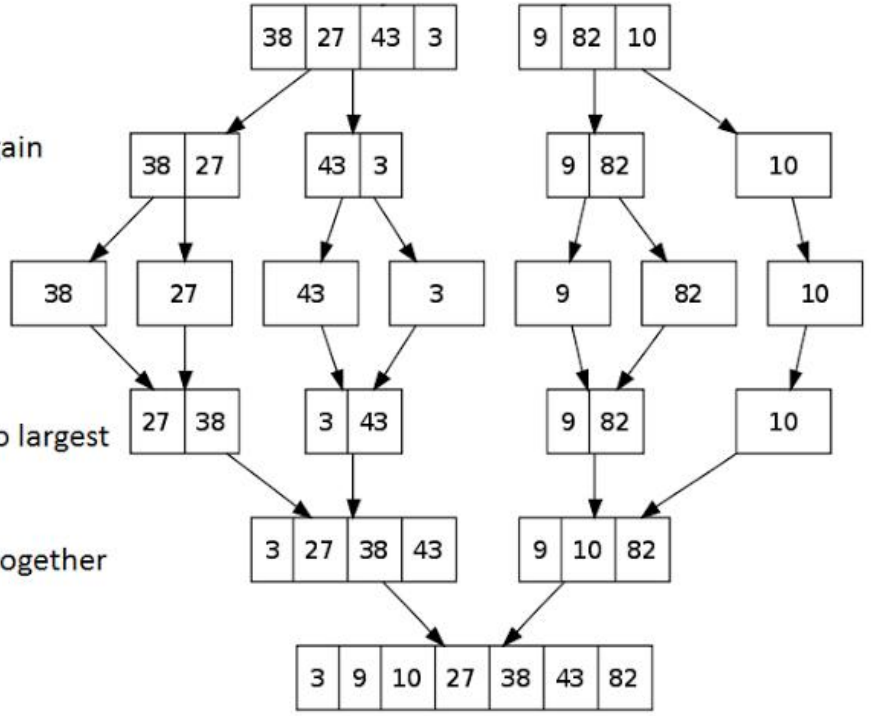


A

1	2	2	3	4	5	6	7

How MergeSort Algorithm Works Internally

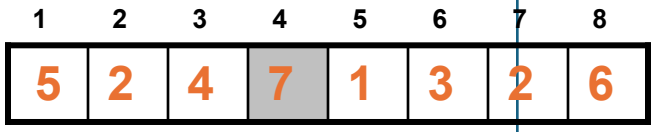
1. Divide the array into two parts
2. Divide the array into two parts again
3. Break each element into single parts
4. Sort the elements from smallest to largest
5. Merge the divided sorted arrays together
6. The array has been sorted



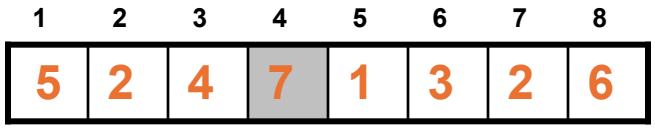
Algorithm of Merge Sort

MERGE-SORT(A, p, r)

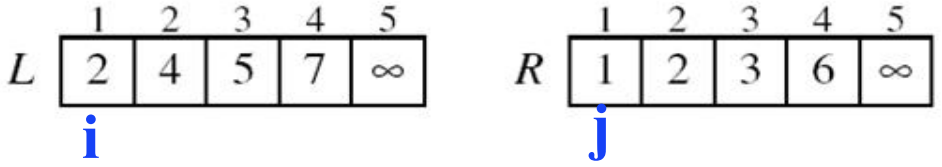
1. if (p < r)
2. q ← $\lfloor (p + r) / 2 \rfloor$
3. **MERGE-SORT(A, p, q)**
4. **MERGE-SORT(A, q + 1, r)**
5. **MERGE(A, p, q, r)**

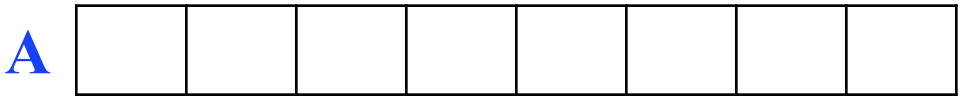


MERGE(A, p, q, r)

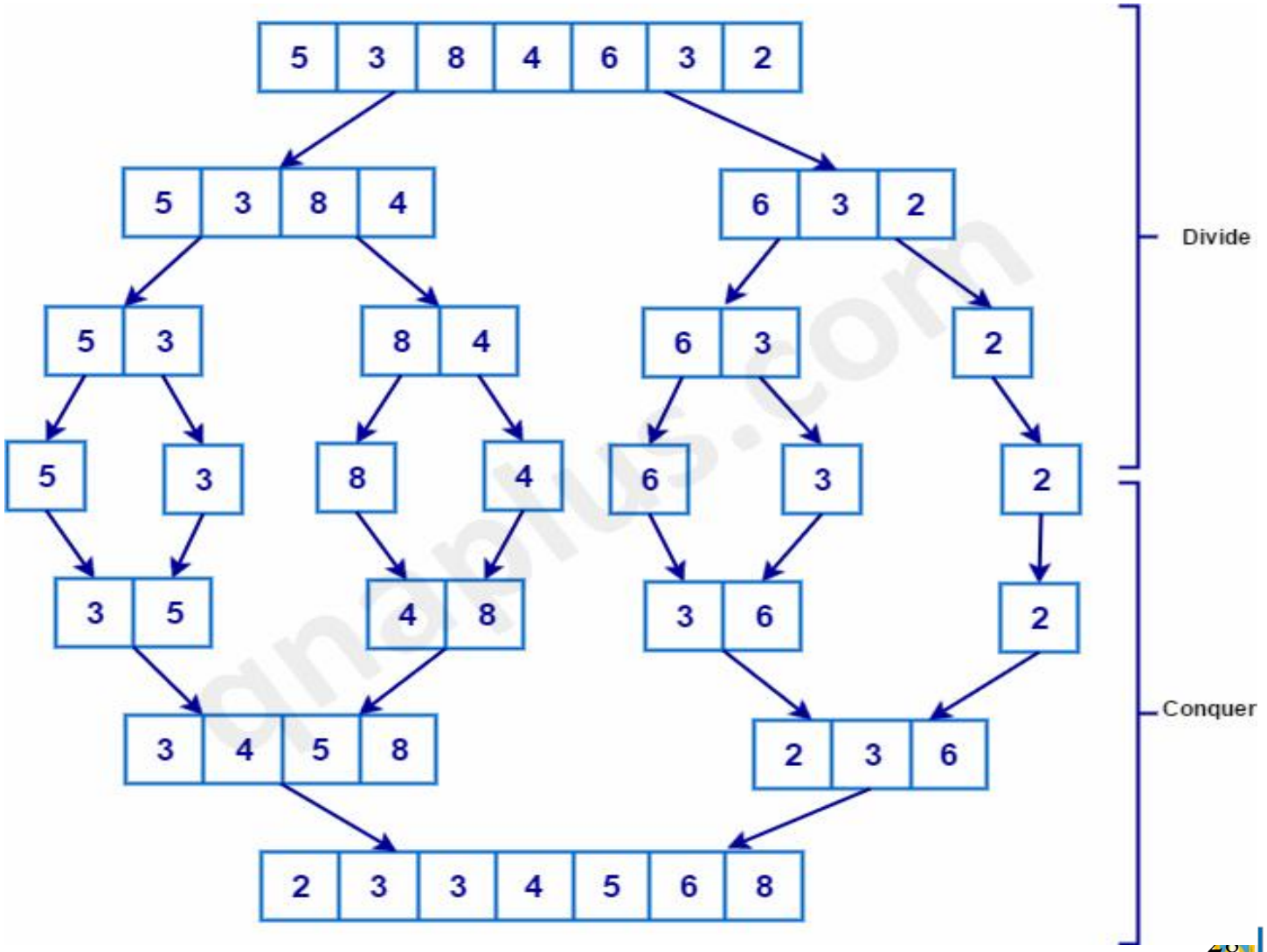


1. n1 = q – p + 1
2. n2 = r - q
3. Let L[1 . . n1 + 1] and R[1 . . n2 + 1] be two new arrays
4. for i = 1 to n1
5. L[i] ← A[p + i - 1]
6. for j = 1 to n2
7. R[j] ← A[q + j]
8. L[n1 + 1] ← ∞
9. R[n2 + 1] ← ∞
10. i ← 1
11. j ← 1
12. for k ← p to r
13. if L[i] ≤ R[j]
14. then i ← i + 1A[k] ← L[i]
15. else A[k] ← R[j]
16. j ← j + 1





Example



Algorithm of Merge Sort

MERGE-SORT(A, p, r)

1. if p < r
2. q ← ⌊(p + r) / 2⌋
3. MERGE-SORT(A, p, q)
4. MERGE-SORT(A, q + 1, r)
5. MERGE(A, p, q, r)

MERGE(A, p, q, r)

n1 = q – p + 1

n2 = r - q

Let L[1 . . n1 + 1] and R[1 . . n2 + 1] be two new arrays

for i = 1 to n1

L[i] ← A[p + i - 1]

for j = 1 to n2

R[j] ← A[q + j]

L[n1 + 1] ← ∞

R[n2 + 1] ← ∞

i ← 1

j ← 1

for k ← p to r

if L[i] ≤ R[j]

then A[k] ← L[i]

i ← i + 1

else A[k] ← R[j]

j ← j + 1

if Remaining elements -> + add that

Time Complexity

MERGE-SORT(A, p, r)

1. if p < r
2. q ← ⌊ (p + r) / 2 ⌋
3. MERGE-SORT(A, p, q) → **T(n/2)**
4. MERGE-SORT(A, q + 1, r) → **T(n/2)**
5. **MERGE(A, p, q, r)** → **O(n)**

Complexity:

$$T(n) = [T(n/2) + T(n/2)] * O(n)$$
$$= 2T(n/2) * O(n)$$
$$= O(\log n) * O(n)$$
$$= O(n \log n)$$

Complexity Analysis of Three Algorithms

Algorithm	Best case	Worst case
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Merge Sort	$O(nlogn)$	$O(nlogn)$

Advantage and Drawbacks

- **Advantages**
  - **Guaranteed to run in O(nlogn)**

- **Disadvantage**
  - **Requires extra space ≈ O(n)**

Question for you

1. Which Algorithm will you choose?
   - For Sorted Data List
   - For Unsorted Data List

2. Is Merge sort a In-Place Algorithm?

3. Sort the given data set using both the Merge Sort. Show each steps.

26	13	6	8	20	22	13

Some References

https://visualgo.net/en/sorting

Reference

- http://coursera.org/learn/algorithms-part1/
- GeeksForGeeks
- YouTube
- LLMs
- Old Lectures of ULAB (Pramanik Sir)

Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh

Department of Computer Science & Engineering